

Lab 9: CUDA Profiling and a Measured Roofline

Stop guessing. Measure a kernel with Nsight, place it on a roofline the tools draw for you, and find out whether the optimisation you were about to do is worth anything.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

DURATION

Two 2-hour blocks

PREREQUISITE

Unit 3, Exercises 17 and 19

SUBMIT

lab09-<ROLLNO>.pdf

MARKS

15

OFFERING

Odd Semester 2026-27

The point of this lab

You have written a tiled matrix multiply and been told it is better. In this lab you find out by how much, why, and where the remaining performance went. You will also discover that one widely repeated claim about tiling is wrong on this hardware, which is the real lesson.

BY THE END OF THIS LAB YOU WILL BE ABLE TO

- Read an Nsight Systems timeline and say whether the GPU is even busy.
- Profile a single kernel with Nsight Compute and read its roofline.
- Compute arithmetic intensity by hand and check it against the measured value.
- Decide, with evidence, whether a kernel is worth optimising further.
- Explain why a textbook speedup figure did not reproduce.

Setup

```
ssh <user>@asai-11-admin  
export PATH=/usr/local/cuda/bin:$PATH  
nvcc --version
```

Everything below runs through the scheduler. Profiling tools are compute; the login node is not for compute.

```
srun --gres=gpu:1 --cpus-per-task=8 --mem=16G --time=00:20:00 --pty bash
```

That gives you an interactive shell **on a GPU node**, which is the comfortable way to work through a profiling session.

Part 1: Is the GPU even busy?

Take your Exercise 19 solution and build a driver that multiplies two 4096×4096 matrices twenty times in a loop.

```
nsys profile -o timeline --stats=true ./matmul_driver
nsys stats timeline.nsys-rep
```

Exercise 1.1. From the summary, report:

- total wall time
- total time in CUDA kernels
- total time in `cudaMemcpy`
- the percentage of wall time the GPU was actually executing a kernel

Exercise 1.2. Your driver copies A and B to the device on every call, because `gpu_matmul_tiled` takes host pointers. Rewrite the driver so the matrices are copied **once** and only the kernel runs twenty times. Report the same four numbers again.

PITFALL · THIS IS THE MOST COMMON REAL-WORLD GPU MISTAKE

If the timeline is mostly transfers and gaps, the kernel is not your problem. Making an already-fast kernel twice as fast recovers nothing when the GPU is idle 70 percent of the time. **Always look at the timeline before optimising a kernel.**

Exercise 1.3. Using your two sets of numbers, compute the effective speedup of the whole application from the transfer change alone. Compare it to the 1.26x that tiling bought. Which change was worth more?

Part 2: One kernel, in detail

```
ncu --set full -o matmul_report ./matmul_driver
ncu --import matmul_report.ncu-rep --page details | head -60
```

Or ask for exactly the two numbers that matter:

```
ncu --metrics \
    sm__throughput.avg.pct_of_peak_sustained_elapsed,\
    dram__throughput.avg.pct_of_peak_sustained_elapsed \
    ./matmul_driver
```

Exercise 2.1. For **both** the naive and the tiled kernel, report:

METRIC	NAIVE	TILED
Duration		
Achieved occupancy		
SM throughput (% of peak)		
DRAM throughput (% of peak)		
L2 hit rate		
Registers per thread		
Shared memory per block		

Exercise 2.2. Look at the **L2 hit rate** of the naive kernel. This is the number that explains the whole lab. Write two sentences on what it implies about how much DRAM traffic tiling is actually saving.

Part 3: The roofline, measured

Nsight Compute draws a roofline for you.

```
ncu --set roofline -o roof ./matmul_driver
```

Open the report and find the roofline chart (`ncu-ui roof.ncu-rep` if you have a display, otherwise export the section as text).

Exercise 3.1. Compute arithmetic intensity **by hand** for both kernels at $n = 4096$:

- Flops: $2n^3$ for both.
- Bytes from DRAM: for the naive kernel assume every element of A and B is read once per output row/column; for the tiled kernel divide by TILE.

State your two numbers in FLOP per byte.

Exercise 3.2. Compare your hand calculation with the intensity Nsight reports. They will not match. Explain the difference. (Hint: your hand calculation assumes no cache. Exercise 2.2 tells you why that assumption fails.)

Exercise 3.3. Where does each kernel sit relative to the ridge point? Is either memory bound? Given the measured position, what would you optimise next, and what would you not bother with?

Part 4: Reproduce the claim, or refute it

The lecture notes state, with measurements taken on one RTX 6000 Ada of this cluster:

VERSION	TIME	GFLOP/S	SPEEDUP
Naive	24.42 ms	5 628	1.00x
Tiled, TILE = 32	19.44 ms	7 071	1.26x
cuBLAS SGEMM	2.17 ms	63 302	11.25x

Exercise 4.1. Reproduce this table on the node you were allocated. State which GPU you got (`nvidia-smi -L`). Do your numbers agree within, say, 15 percent? If not, say what differs.

Exercise 4.2. Add a cuBLAS row of your own:

```
cublasHandle_t h; cublasCreate(&h);
float alpha = 1.0f, beta = 0.0f;
/* cuBLAS is column-major; passing B and A in this order yields row-major A*B */
cublasSgemm(h, CUBLAS_OP_N, CUBLAS_OP_N, n, n, n,
            &alpha, dB, n, dA, n, &beta, dC, n);
```

Compile with `-lcublas`. Verify the result agrees with your tiled kernel before believing the timing.

Exercise 4.3. Most textbooks claim tiling gives roughly a ten-fold speedup. You will measure something closer to 1.3x. Give the two hardware reasons, and support each with a number from your own profile.

BEYOND THE SYLLABUS · THIS IS THE ACTUAL SKILL

Anyone can copy an optimisation from a book. The professional skill is measuring whether it did anything on the machine in front of you, and being able to say why when it did not. A performance claim without a measurement and a machine description is worthless, however respectable its source.

Part 5: Vary something

Exercise 5.1. Change `TILE` to 8, 16 and 32 and measure. Plot GFLOP/s against tile size. Explain the shape using the shared-memory-per-block and occupancy figures from your Part 2 table.

Exercise 5.2. Remove **one** of the two `__syncthreads()` calls. Run the correctness test five times.

- Did it fail every time, sometimes, or never?
- Does the failure rate change with matrix size?
- What does this tell you about testing concurrent code once and calling it correct?

Put the barrier back.

What to submit

lab09-<ROLLNO>.pdf :

1. The Part 1 timeline numbers, before and after the transfer fix.
2. The completed Part 2 metric table for both kernels.
3. Your hand-computed intensities and the reconciliation with the measured ones.
4. Your reproduction of the performance table, with the GPU you were allocated named, plus your cuBLAS row.
5. The tile-size plot.
6. Your Exercise 5.2 observations.

MARKS	FOR
3	Part 1, including the effective-speedup comparison in 1.3
4	Part 2 metrics, correctly read, with the L2 interpretation
3	Part 3, hand calculation reconciled with the measurement
3	Part 4, table reproduced and the discrepancy explained with evidence
2	Part 5, tile-size sweep and the barrier experiment

ON ASAI COMPUTE · NOTE ON TOOL AVAILABILITY

`nsys` and `ncu` ship with the CUDA toolkit at `/usr/local/cuda/bin`. Some counters need elevated permissions; if `ncu` reports that it cannot access performance counters, use `--set basic` and note the limitation in your report rather than abandoning the exercise. Reporting what you could not measure, and why, is part of doing this properly.